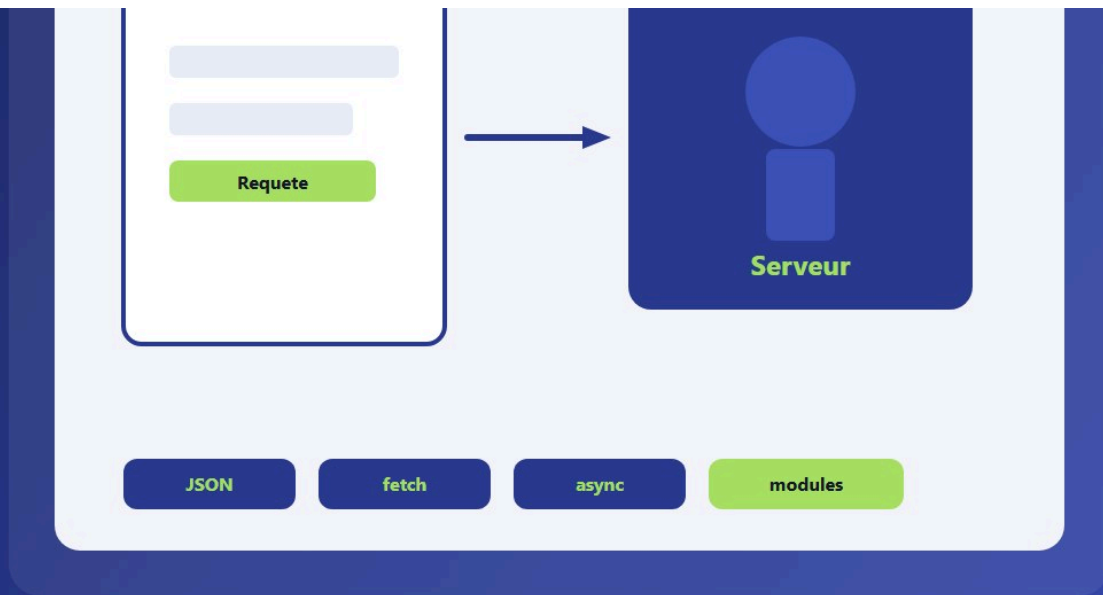




FORMATION INFORMATIQUE - SUITE

JavaScript - La suite

Formulaires, JSON, fetch, async/await et modules : la suite solide apres les bases.

DanielCraft - Livre debutant

Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

1. [Chapitre 1 - Rappel rapide et carte du livre](#)

- [Ce que tu gardes en tete](#)
- [La carte du livre](#)
- [Un fil rouge](#)
- [Comment lire ce livre](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

2. [Chapitre 2 - Formulaire qui tiennent la route](#)

- [Lire les champs](#)
- [Valider sans etre mechant](#)
- [HTML5 aide, JS decide](#)
- [Cas concret : todo avec garde-fou](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

3. [Chapitre 3 - JSON, le format des donnees voyageuses](#)

- [A quoi ca ressemble](#)
- [De texte vers objet : parse](#)
- [D'objet vers texte : stringify](#)
- [Pieges courants](#)
- [Exemple concret : todo en texte](#)
- [En vrai](#)
- [A toi](#)

4. [Chapitre 4 - fetch GET : demander des infos](#)

- [L'idee en une image](#)
- [Premier appel](#)
- [Afficher dans la page](#)
- [Meteo : autre exemple mental](#)
- [Ce que fetch renvoie vraiment](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

5. [Chapitre 5 - Les promesses, sans jargon opaque](#)

- [Image mentale](#)
- [then : quand ca marche](#)
- [catch : quand ca casse](#)
- [Pourquoi pas juste "attendre" ?](#)
- [Chainer sans se noyer](#)
- [Ce que DanielCraft veut que tu sentes](#)

- [Erreur classique](#)
 - [En vrai](#)
 - [A toi](#)
6. [Chapitre 6 - async/await : du fetch lisible](#)
- [Le geste de base](#)
 - [Reecrire l'exemple meteo](#)
 - [Attention : await seulement dans async](#)
 - [try/catch avec await](#)
 - [Quand garder then ?](#)
 - [Erreur classique](#)
 - [En vrai](#)
 - [A toi](#)
7. [Chapitre 7 - Erreurs reseau et reponses foireuses](#)
- [response.ok](#)
 - [try/catch async, version complete](#)
 - [Différents types d'echecs](#)
 - [Ne mens pas a l'utilisateur](#)
 - [Cas meteo](#)
 - [Erreur classique](#)
 - [En vrai](#)
 - [A toi](#)
8. [Chapitre 8 - fetch POST : envoyer un peu de JSON](#)
- [Anatomie d'un POST](#)
 - [Brancher sur un formulaire](#)
 - [GET vs POST en une phrase](#)
 - [Ce que le serveur renvoie](#)
 - [Erreur classique](#)
 - [En vrai](#)
 - [A toi](#)
9. [Chapitre 9 - Modules : decouper le code](#)
- [Un exemple simple](#)
 - [Pieges a connaitre](#)
 - [En vrai](#)
 - [A toi](#)
10. [Chapitre 10 - Organiser un petit projet](#)
- [Responsabilites](#)
 - [Noms clairs](#)
 - [Une seule source de verite](#)
 - [En vrai](#)
 - [A toi](#)
11. [Chapitre 11 - Deboguer mieux](#)
- [La console, ton amie](#)
 - [Lire une stack](#)

- [Points d'arrêt](#)
 - [Hypotheses courtes](#)
 - [En vrai](#)
 - [A toi](#)
12. [Chapitre 12 - Mini-projet : liste depuis une API](#)
- [Parcours](#)
 - [Squelette](#)
 - [Variante meteo \(idee\)](#)
 - [Qualite minimale](#)
 - [A toi](#)
13. [Chapitre 13 - Ce qu'il faut retenir](#)
- [En une minute](#)
 - [Habitudes solides](#)
 - [Erreurs classiques](#)
 - [Suite dans ce livre](#)
 - [Mini defi](#)
14. [Chapitre 14 - Atelier : formulaire robuste](#)
- [Etapas](#)
 - [Criteres de reussite](#)
 - [Bonus](#)
 - [Piege](#)
15. [Chapitre 15 - Atelier : fetch et afficher](#)
- [Preparation](#)
 - [Etapas](#)
 - [Criteres de reussite](#)
 - [Bonus](#)
16. [Chapitre 16 - Atelier : decouper en modules](#)
- [Fichiers](#)
 - [Etapas](#)
 - [Criteres de reussite](#)
 - [Bonus](#)
17. [Chapitre 17 - CORS, en une page claire](#)
- [L'idee simple](#)
 - [Ce que tu peux faire en apprenant](#)
 - [Ce qu'il ne faut pas croire](#)
 - [En vrai](#)
 - [A toi](#)
18. [Chapitre 18 - Debounce : ne pas frapper trop vite](#)
- [Idee en code](#)
 - [Quand l'utiliser](#)
 - [Quand ne pas l'utiliser](#)
 - [En vrai](#)

- [A toi](#)

19. [Chapitre 19 - Petites habitudes de performance](#)

- [Moins de travail inutile](#)
- [Dom : batch mental](#)
- [Reseau](#)
- [Code lisible d'abord](#)
- [En vrai](#)
- [A toi](#)

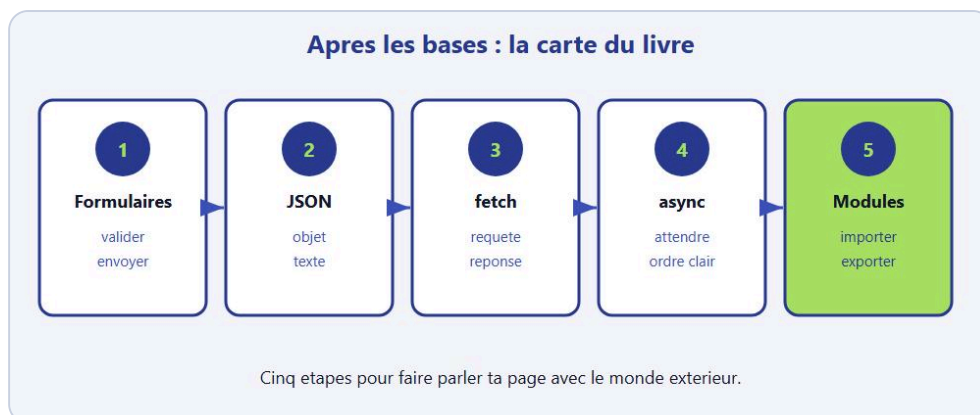
20. [Quiz final](#)

- [Questions](#)
- [Corriges](#)

21. [Bravo](#)

- [Ce que tu sais faire maintenant](#)
- [La suite possible](#)
- [Un conseil pour la route](#)
- [A toi](#)

Chapitre 1 - Rappel rapide et carte du livre



Après les bases : formulaires, JSON, fetch, async, modules.

Tu as déjà fait les bases. Variables, conditions, boucles, fonctions, DOM, evenements. Si quelque chose est flou, ce n'est pas grave : on ne va pas tout refaire. Ce livre part du moment où la page commence à parler avec le monde extérieur.

Chez DanielCraft, on aime bien une image simple. Les bases, c'était apprendre à faire bouger une pièce. La suite, c'est ouvrir la fenêtre, envoyer un message, attendre une réponse, et ranger le code pour qu'il reste lisible.

Ce que tu gardes en tête

Tu sais sélectionner un élément avec `querySelector`. Tu sais écouter un clic ou un `submit`. Tu sais changer un texte avec `textContent`. Tu sais écrire une fonction. C'est assez pour avancer.

On ne revient pas sur `let`, `if`, `for`, ni sur "c'est quoi le DOM". Si tu bloques sur ces mots, relis le premier livre. Ici, on monte d'un cran.

La carte du livre

Imagine une petite appli météo, un formulaire de contact, ou une liste de produits. Pour que ça marche vraiment, tu as besoin de plusieurs pièces.

D'abord les formulaires. Tu récupères ce que la personne a tapé, tu vérifies, tu affiches une erreur claire si besoin. Ensuite JSON : un format de texte pour transporter des données (nom, prix, température...). Puis `fetch` : ta page demande des infos à un serveur. Les promesses et `async/await` t'aident à attendre la réponse sans te perdre. Les modules te permettent de découper le code en fichiers. Et à la fin, tu débogues, tu organises, tu fais de petits ateliers.

Un fil rouge

On va souvent parler de trois exemples. Une todo un peu plus sérieuse. Une météo qui charge des données. Un formulaire de contact qui valide avant d'envoyer. Ces exemples reviennent, pour que tu sentes le progrès.

Comment lire ce livre

Lis dans l'ordre au debut. Formulaires, JSON, fetch, promesses, async : ca s'enchaîne. Ensuite modules et organisation. Les ateliers sont la pour faire, pas seulement lire. Le quiz sert a verifier. Le dernier chapitre, c'est juste pour souffler et voir la suite.

Tu n'as pas besoin d'un framework. React, Vue, et les autres viendront plus tard si tu veux. Ici, on reste en JavaScript "nu", clair, dans le navigateur.

Erreur classique

Croire que "je connais les bases" = "je sais faire une vraie app". Les bases sont le moteur. La suite, c'est la route, le carburant, et la carte. Sans fetch et JSON, ta page reste seule dans sa bulle. Sans modules, ton fichier devient un spaghetti. Sans validation de formulaire, tu envoies n'importe quoi.

En vrai

Ouvre une page que tu as déjà codée (compteur, todo simple...). Note ce qui manque pour en faire quelque chose d'utile : charger une liste depuis ailleurs ? envoyer un message ? découper le fichier ? Ce livre répond a ça.

A toi

Ecris en trois phrases ce que tu veux construire a la fin. Pas un reseau social. Quelque chose de petit : "afficher la meteo de ma ville", "valider un formulaire contact", "charger une liste de produits". Garde ce but. On y reviendra.

Chapitre 2 - Formulaires qui tiennent la route

Valider avant d'envoyer, expliquer l'erreur clairement.

Un formulaire, c'est une conversation. La personne écrit. Ta page écoute. Si quelque chose cloche, tu le dis clairement. Si tout va bien, tu continues.

Beaucoup de débutants font deux erreurs. Soit ils laissent le navigateur tout gérer sans rien expliquer. Soit ils envoient le formulaire sans vérifier, et la page se recharge en silence. Ici, on prend le contrôle.

Lire les champs

Imagine un petit contact : nom, email, message.

```
<form id="contact">
  <label for="nom">Nom</label>
  <input id="nom" name="nom" type="text">

  <label for="email">Email</label>
  <input id="email" name="email" type="email">

  <label for="message">Message</label>
  <textarea id="message" name="message"></textarea>

  <p id="erreur" hidden></p>
  <button type="submit">Envoyer</button>
</form>
```

En JS, tu sélectionnes le formulaire et tu écoutes `submit`.

```
const form = document.querySelector("#contact");
const zoneErreur = document.querySelector("#erreur");

form.addEventListener("submit", function (event) {
  event.preventDefault();

  const nom = form.nom.value.trim();
  const email = form.email.value.trim();
  const message = form.message.value.trim();

  // on validera juste après
});
```

`preventDefault()` dit au navigateur : "ne recharge pas, je gère". Sans ça, ta page clignote et ton message disparaît. C'est le geste numéro un des formulaires en JS.

Valider sans etre mechant

La validation, ce n'est pas punir. C'est aider. Un message utile dit quoi corriger. Un message inutile dit juste "erreur".

Exemple simple :

```
function afficherErreur(texte) {
  zoneErreur.hidden = false;
  zoneErreur.textContent = texte;
}

function cacherErreur() {
  zoneErreur.hidden = true;
  zoneErreur.textContent = "";
}

form.addEventListener("submit", function (event) {
  event.preventDefault();
  cacherErreur();

  const nom = form.nom.value.trim();
  const email = form.email.value.trim();
  const message = form.message.value.trim();

  if (nom === "") {
    afficherErreur("Dis-nous ton nom, meme un prenom suffit.");
    form.nom.focus();
    return;
  }

  if (email === "" || !email.includes("@")) {
    afficherErreur("L'email semble incomplet. Verifie le @.");
    form.email.focus();
    return;
  }

  if (message.length < 10) {
    afficherErreur("Ton message est un peu court. Ajoute quelques mots.");
    form.message.focus();
    return;
  }

  // Ici, tout est ok : on pourrait envoyer plus tard avec fetch
  zoneErreur.hidden = false;
  zoneErreur.textContent = "Merci " + nom + " ! Message pret a partir.";
});
```

Tu vois l'idée. On arrête dès la première erreur. On remet le focus sur le champ. On explique en français simple. Chez DanielCraft, on préfère un message humain à un code d'erreur mystérieux.

HTML5 aide, JS decide

Les attributs `required`, `type="email"`, `minlength` aident. Mais ne compte pas seulement dessus. Les navigateurs ne montrent pas tous les mêmes bulles. Et parfois tu as des règles métier ("le message doit parler d'un produit", "le stock doit être un nombre positif"). JS reste le chef d'orchestre.

Tu peux combiner : laisser HTML pour le minimum, puis affiner en JS.

Cas concret : todo avec garde-fou

Sur une todo, le piège classique c'est d'ajouter une tâche vide.

```
const champ = document.querySelector("#tache");
const liste = document.querySelector("#liste");
const formTodo = document.querySelector("#form-todo");

formTodo.addEventListener("submit", function (event) {
  event.preventDefault();
  const texte = champ.value.trim();

  if (texte === "") {
    afficherErreur("Ecris une tâche avant d'ajouter.");
    return;
  }

  const li = document.createElement("li");
  li.textContent = texte;
  liste.appendChild(li);
  champ.value = "";
  champ.focus();
  cacherErreur();
});
```

Même logique partout : lire, nettoyer (`trim`), vérifier, agir.

Erreur classique

Afficher l'erreur une seule fois, puis ne jamais la cacher. La personne corrige, renvoie, et l'ancien message reste. Moche. Efface l'erreur au début de chaque tentative, ou dès que la personne retape dans le champ.

Autre piège : valider seulement au clic du bouton, jamais au `submit`. Si quelqu'un appuie sur Entrée, ton code ne part pas. Écoute `submit` sur le formulaire. C'est plus solide.

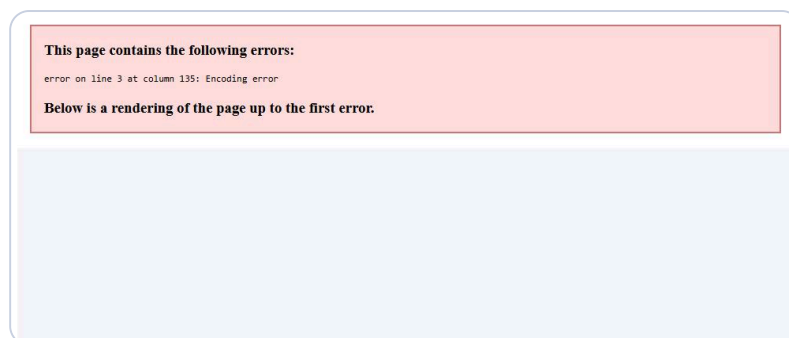
En vrai

Ouvre un formulaire de site (inscription, contact...). Essaie d'envoyer vide. Regarde le message. Est-il clair ? Est-ce qu'il pointe le champ ? Note ce que tu aimes. Tu peux voler les bonnes idées pour ton propre code.

A toi

Fais un formulaire "avis produit" : note de 1 à 5, commentaire. Refuse une note hors plage. Refuse un commentaire de moins de 5 caractères. Affiche un message de succès quand tout est bon. Pas besoin d'envoyer au serveur pour l'instant.

Chapitre 3 - JSON, le format des donnees voyageuses



JSON = donnees rangees en texte, lisibles par la machine.

Quand ta page parle a un serveur, elle n'envoie pas un objet JavaScript magique. Elle envoie du texte. JSON, c'est une facon d'ecrire ce texte pour que tout le monde se comprenne : navigateur, serveur, API meteo, boutique en ligne...

JSON veut dire "JavaScript Object Notation". Le nom fait peur. L'idee est simple : des donnees rangees comme des objets et des tableaux, mais en texte.

A quoi ca ressemble

Voici une fiche produit en JSON :

```
{
  "nom": "Casque audio",
  "prix": 59.9,
  "enStock": true,
  "tags": ["son", "promo"]
}
```

Et une liste :

```
[
  { "ville": "Paris", "temp": 18 },
  { "ville": "Lyon", "temp": 16 }
]
```

Tu reconnais les objets `{ }`, les tableaux `[ ]`, les chaines entre guillemets doubles, les nombres, `true` / `false`, et `null`. En JSON "strict", les cles sont toujours entre guillemets doubles. Pas de fonction. Pas de commentaire.

De texte vers objet : parse

Ta page recoit souvent une chaine. Pour travailler avec, tu la transformes en vrai objet JS.

```
const texte = '{"nom":"Casque audio","prix":59.9}';
const produit = JSON.parse(texte);

console.log(produit.nom); // Casque audio
console.log(produit.prix); // 59.9
```

`JSON.parse` lit le texte et construit l'objet. Si le texte est casse, ça plante. On verra les erreurs plus loin.

D'objet vers texte : `stringify`

L'inverse : tu as un objet JS, tu veux l'envoyer ou le stocker.

```
const contact = {
  nom: "Lea",
  email: "lea@exemple.fr",
  message: "Bonjour, je veux un devis."
};

const texte = JSON.stringify(contact);
console.log(texte);
// {"nom":"Lea","email":"lea@exemple.fr","message":"Bonjour, je veux un devis."}
```

Utile pour `fetch` en POST, pour `localStorage`, pour un fichier... DanielCraft utilise souvent ce reflexe : objet en memoire pour travailler, JSON en texte pour voyager.

Pieges courants

Premier piege : confondre objet JS et JSON. En JS, tu peux ecrire `{ nom: "Lea" }` avec des guillemets simples autour des valeurs, et parfois sans guillemets sur les cles. En JSON "vrai", les cles et les chaines sont en guillemets doubles.

Deuxieme piege : `JSON.parse` sur quelque chose qui n'est pas du JSON.

```
JSON.parse("salut"); // erreur
JSON.parse("{nom: Lea}"); // erreur (pas de guillemets)
```

Troisieme piege : croire que `stringify` garde les fonctions. Non. Les fonctions disparaissent. Les dates deviennent des chaines. Les `undefined` sautent souvent. JSON transporte des donnees, pas du comportement.

Quatrieme piege : parser deux fois, ou stringifier deux fois, et se retrouver avec une chaine bizarre.

```
const dejaTexte = '{"a":1}';
const encore = JSON.stringify(dejaTexte);
// result: "\"{\\\"a\\\":1}\"" -> un vrai bordel
```

Regle simple : parse quand tu recois du texte JSON. Stringify quand tu veux produire du texte JSON. Une fois suffit.

Exemple concret : todo en texte

```
const taches = [
  { id: 1, titre: "Acheter du pain", faite: false },
  { id: 2, titre: "Appeler Lea", faite: true }
];

const pourStockage = JSON.stringify(taches);
// plus tard...
const relues = JSON.parse(pourStockage);
console.log(relues[0].titre); // Acheter du pain
```

Tu peux imaginer la meme chose avec une reponse meteo ou une liste de produits d'une API.

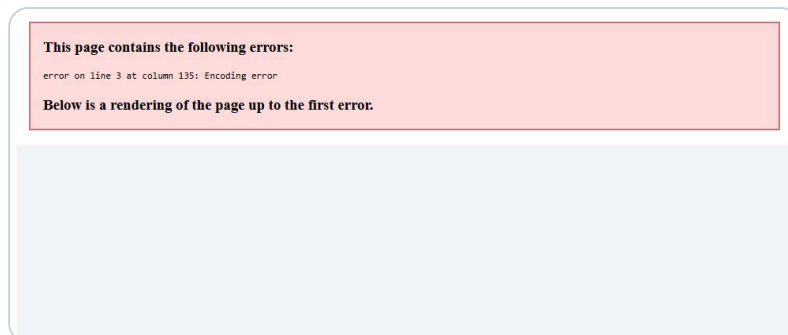
En vrai

Ouvre la console du navigateur. Colle un petit `JSON.parse('{ "ville": "Nantes", "temp": 14 }')`. Affiche `ville` et `temp`. Puis `JSON.stringify` un objet de ton choix. Regarde le texte produit. C'est tout. Mais ce geste-la, tu vas le refaire cent fois.

A toi

Cree un objet `commande` avec client, produits (tableau de noms), et total. Transforme-le en texte JSON. Reparse-le. Affiche une phrase : "Commande de X pour Y euros". Si ca marche, JSON n'a plus de mystere.

Chapitre 4 - fetch GET : demander des infos



Ta page demande des infos a un serveur, puis affiche la reponse.

Jusqu'ici, tes donnees vivaient dans ton fichier. Maintenant, ta page peut demander des infos ailleurs : une API meteo, une liste de produits, un fichier JSON sur un serveur.

`fetch` est l'outil moderne du navigateur pour ca. En mode GET (le mode par default), tu dis : "donne-moi cette ressource".

L'idée en une image

Ta page envoie une demande a une adresse (une URL). Le serveur repond. Tu lis la reponse. Souvent, la reponse est du JSON. Tu le parses (souvent avec `.json()`), puis tu affiches.

Chez DanielCraft, on resume comme ca : demander, attendre, lire, montrer.

Premier appel

Il existe des API publiques pour s'entrainer. L'exemple ci-dessous utilise une URL fictive pour rester clair. Le schema reste le meme partout.

```
fetch("https://exemple.api/produits")
  .then(function (reponse) {
    return reponse.json();
  })
  .then(function (produits) {
    console.log(produits);
  });
```

Que se passe-t-il ? `fetch` part chercher l'URL. Quand la reponse HTTP arrive, le premier `then` recoit un objet `Response`. `reponse.json()` lit le corps et le transforme en objet/tableau JS (comme un `JSON.parse` intelligent). Le second `then` recoit les donnees pretes.

On detaillera les promesses au chapitre suivant. Pour l'instant, retiens la chaine : `fetch -> json -> utiliser`.

Afficher dans la page

Imagine une liste de produits.

```
<ul id="liste"></ul>
<p id="statut">Chargement...</p>
```

```
const liste = document.querySelector("#liste");
const statut = document.querySelector("#statut");

fetch("https://exemple.api/produits")
  .then(function (reponse) {
    return reponse.json();
  })
  .then(function (produits) {
    statut.textContent = produits.length + " produits trouves";
    liste.innerHTML = "";

    for (const p of produits) {
      const li = document.createElement("li");
      li.textContent = p.nom + " - " + p.prix + " €";
      liste.appendChild(li);
    }
  });
```

Tu vois le pattern. Pendant le chargement, tu peux montrer un message. Ensuite tu vides la liste et tu ajoutes les elements. C'est le coeur de beaucoup d'apps web simples.

Meteo : autre exemple mental

Tu appelles une URL avec une ville. Tu recois `{ "ville": "Paris", "temp": 18, "ciel": "nuageux" }`. Tu ecris dans un `p` : "A Paris, il fait 18°C, ciel nuageux." Memes etapes. Seules les proprietes changent.

Ce que fetch renvoie vraiment

Attention : `fetch` reussit souvent meme si le serveur dit "404 introuvable". Techniquement, la requete reseau a marche. Le code HTTP peut etre une erreur. On traitera `response.ok` au chapitre erreurs. Pour ce chapitre, concentre-toi sur le chemin heureux : URL ok, JSON ok, affichage ok.

Erreur classique

Oublier `return reponse.json()`. Si tu ecris seulement `reponse.json()` sans `return` dans le `then`, le suivant recoit `undefined`. Autre classique : traiter la reponse comme du JSON alors que c'est du HTML d'erreur. Ou appeler une mauvaise URL et croire que "fetch est casse".

En vrai

Cherche une API publique simple (JSONPlaceholder, Open-Meteo, etc. selon ce qui est dispo chez toi). Ouvre l'URL dans le navigateur. Regarde le JSON brut. Puis tente un petit `fetch` dans la console sur une page autorisee, ou dans un fichier local servi correctement. Le but : voir des vraies donnees arriver.

A toi

Ecris une fonction `chargerProduits(url)` qui fait un GET, lit le JSON, et affiche chaque `nom` dans une liste. Meme avec des donnees fictives en local (un fichier `produits.json` a cote), le geste compte. Si tu n'as pas encore de serveur local, ecris le code quand meme : tu le brancheras au prochain chapitre avec `async/await`.

Chapitre 5 - Les promesses, sans jargon opaque

Quand tu appelles `fetch`, le resultat n'arrive pas tout de suite. Le reseau prend du temps. JavaScript ne peut pas rester bloqué à attendre comme un ascenseur en panne : la page doit rester utilisable.

Une promesse, c'est un objet qui dit : "je te donnerai une valeur plus tard, ou une erreur". Tu branches ensuite ce qui se passe en cas de succès (`then`) ou d'échec (`catch`).

Image mentale

Tu commandes une pizza. Le restaurant te donne un ticket. Ce ticket n'est pas la pizza. C'est la promesse de la pizza. Plus tard, soit tu reçois la pizza (`then`), soit on t'annonce un problème (`catch`).

`fetch` te rend ce ticket tout de suite. Les données viennent après.

then : quand ça marche

```
fetch("https://exemple.api/meteo?ville=Paris")
  .then(function (reponse) {
    return reponse.json();
  })
  .then(function (data) {
    console.log("Temperature :", data.temp);
  });
```

Chaque `then` peut renvoyer une nouvelle valeur. Le `then` suivant la reçoit. C'est pour ça qu'on écrit `return reponse.json() : json()` renvoie aussi une promesse (parce que lire le corps prend un peu de temps).

catch : quand ça casse

```
fetch("https://exemple.api/meteo?ville=Paris")
  .then(function (reponse) {
    return reponse.json();
  })
  .then(function (data) {
    console.log(data.temp);
  })
  .catch(function (erreur) {
    console.log("Probleme :", erreur.message);
  });
```

Si le réseau tombe, ou si `json()` explose sur un texte invalide, tu tombes dans `catch`. Un seul `catch` à la fin peut couvrir la chaîne. Pratique.

Pourquoi pas juste "attendre" ?

Parce que JS dans le navigateur est événementiel. Pendant que la requête voyage, l'utilisateur peut encore cliquer, scroller, écrire. Les promesses permettent de dire "continue, et rappelle-moi quand c'est prêt".

Tu n'as pas besoin de connaître tous les états internes (`pending`, `fulfilled`, `rejected`) pour commencer. Retiens : une promesse aboutit ou échoue ; tu branches `then` et `catch`.

Chainer sans se noyer

Evite les pyramides. Preferes une ligne claire :

```
function afficherMeteo(ville) {
  return fetch("https://exemple.api/meteo?ville=" + encodeURIComponent(ville))
    .then(function (reponse) {
      return reponse.json();
    })
    .then(function (data) {
      document.querySelector("#meteo").textContent =
        "A " + data.ville + ", " + data.temp + "°C";
    })
    .catch(function () {
      document.querySelector("#meteo").textContent =
        "Impossible de charger la meteo.";
    });
}

afficherMeteo("Lyon");
```

`encodeURIComponent` protege les accents et espaces dans l'URL. Petit detail utile.

Ce que DanielCraft veut que tu sentes

Les promesses ne sont pas un exercice theorique. C'est le langage du "plus tard" sur le web. Fetch, fichiers, timers avances, plein d'APIs modernes parlent en promesses. Une fois l'idee du ticket pizza digeree, `async/await` (chapitre suivant) devient presque confortable.

Erreur classique

Oublier le `catch`, puis ne rien voir quand ca rate. Ou mettre du code juste apres `fetch(...)` en croyant que les donnees sont deja la.

```
// FAUX reflexe
fetch(url);
console.log("deja arrive ?"); // non, trop tot
```

Le code apres `fetch` s'execute tout de suite. Les donnees, elles, arrivent dans `then`.

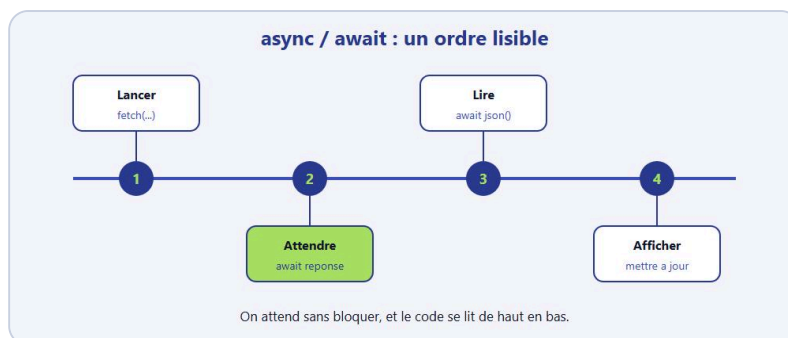
En vrai

Prends un `fetch` qui marche. Ajoute un `catch` qui ecrit un message dans la page. Coupe le wifi une seconde (ou mets une URL inventee) et regarde le `catch` se declencher. Tu dois sentir la difference entre "succes" et "echec".

A toi

Ecris `chargerTodos(url)` avec `then` / `catch`. En succes, affiche le nombre de todos. En echec, affiche "Liste indisponible". Pas besoin que l'API soit parfaite : le branchement compte.

Chapitre 6 - async/await : du fetch lisible



async/await : attendre sans bloquer, dans un ordre lisible.

Les `then` marchent. Mais quand tu enchaînes plusieurs étapes, ça devient vite une chenille. `async` et `await` permettent d'écrire presque comme du code "normal", ligne après ligne, tout en restant asynchrone.

Le geste de base

Tu marques une fonction avec `async`. Dedans, tu peux `await` une promesse. JavaScript attend le résultat de cette ligne, puis continue.

```

async function chargerProduits() {
  const reponse = await fetch("https://exemple.api/produits");
  const produits = await reponse.json();
  console.log(produits);
}

chargerProduits();

```

Compare avec la version `then`. C'est la même histoire, plus lisible. Chez DanielCraft, on passe souvent en `async/await` dès que le flux dépasse deux étapes.

Recrire l'exemple meteo

Version claire :

```

async function afficherMeteo(ville) {
  const url = "https://exemple.api/meteo?ville=" + encodeURIComponent(ville);
  const reponse = await fetch(url);
  const data = await reponse.json();

  document.querySelector("#meteo").textContent =
    "A " + data.ville + ", il fait " + data.temp + "°C";
}

```

Tu lis de haut en bas. Demande. Lis JSON. Affiche. Le cerveau aime ça.

Attention : await seulement dans async

Tu ne peux pas écrire `await fetch(...)` tout seul au milieu d'un script classique (sauf modules top-level dans certains contextes modernes, on n'en fait pas une règle ici). Encapsule dans `async function ...`.

Aussi : une fonction `async` renvoie toujours une promesse. Si tu `return produits`, l'appelant reçoit une promesse de produits. Il devra `await` ou `.then` à son tour.

```
async function getProduits() {
  const reponse = await fetch("https://exemple.api/produits");
  return reponse.json();
}

async function demarrer() {
  const produits = await getProduits();
  console.log(produits.length);
}

demarrer();
```

try/catch avec await

Les erreurs, on les détaille au chapitre suivant. Mais le réflexe arrive déjà :

```
async function afficherMeteo(ville) {
  try {
    const reponse = await fetch(
      "https://exemple.api/meteo?ville=" + encodeURIComponent(ville)
    );
    const data = await reponse.json();
    document.querySelector("#meteo").textContent =
      data.ville + " : " + data.temp + "°C";
  } catch (erreur) {
    document.querySelector("#meteo").textContent =
      "Meteo indisponible pour le moment.";
  }
}
```

`try/catch` autour d'`await`, c'est l'équivalent confortable de `.catch`.

Quand garder then ?

Parfois un tout petit enchaînement. Parfois une lib qui renvoie des promesses et tu branches un seul `then`. Ce n'est pas interdit. Mais pour du code que tu lis dans six mois, `async/await` gagne souvent.

Erreur classique

Oublier `await` :

```
const reponse = fetch(url); // reponse = Promise, pas Response
const data = await reponse.json(); // plante : Promise n'a pas .json
```

Ou croire que `async` magique rend le code synchrone pour tout le monde. Non : ça rend ton écriture linéaire. Le temps, lui, continue de passer ailleurs.

En vrai

Reprends un `fetch` en `then` que tu as déjà. Reécris-le en `async/await`. Compare les deux versions côte à côte. Laquelle te parle le plus ? Garde celle-là pour la suite du livre.

A toi

Ecris `async function chargerEtAfficher(url)` qui charge une liste JSON et remplit un `<ul>`. Ajoute un message "Chargement..." avant le fetch, puis remplace-le par le resultat. Tu dois sentir le rythme : avant / pendant / apres.

Chapitre 7 - Erreurs reseau et reponses foireuses

Le reseau n'est pas gentil. Wifi coupe. Serveur en vacances. URL tapee de travers. JSON invalide. Et le piege le plus vicieux : une reponse HTTP 404 ou 500 que `fetch` considere quand meme comme "arrivee".

Ce chapitre te donne un reflexe solide : verifier, attraper, expliquer a l'utilisateur.

`response.ok`

Quand `fetch` se termine sans planter, tu as un objet `Response`. Regarde `reponse.ok` : c'est `true` pour les codes 200-299. Sinon, quelque chose cloche cote serveur ou URL.

```
async function chargerProduits(url) {
  const reponse = await fetch(url);

  if (!reponse.ok) {
    throw new Error("HTTP " + reponse.status);
  }

  return reponse.json();
}
```

Tu transformes un "mauvais statut" en vraie erreur. Ensuite `try/catch` peut la recuperer. C'est le pattern que DanielCraft recommande presque toujours avec fetch.

`try/catch async, version complete`

```
const statut = document.querySelector("#statut");
const liste = document.querySelector("#liste");

async function afficherProduits() {
  statut.textContent = "Chargement...";
  liste.innerHTML = "";

  try {
    const reponse = await fetch("https://exemple.api/produits");

    if (!reponse.ok) {
      throw new Error("Le serveur a repondu " + reponse.status);
    }

    const produits = await reponse.json();

    if (!Array.isArray(produits)) {
      throw new Error("Format inattendu");
    }

    statut.textContent = produits.length + " produits";
    for (const p of produits) {
      const li = document.createElement("li");
      li.textContent = p.nom;
      liste.appendChild(li);
    }
  } catch (erreur) {
    statut.textContent = "Impossible de charger la liste. Reessaie plus tard.";
    console.error(erreur);
  }
}
```

```
}  
  
afficherProduits();
```

Tu vois les couches. Message pendant le chargement. Verification HTTP. Verification du format. Affichage. Et si ca casse, un message humain + le detail technique dans la console pour toi.

Differents types d'echecs

Coupure reseau : `fetch` rejette, tu tombes dans `catch`.

404 / 500 : `fetch` resout, mais `ok` est false. D'ou le `if (!reponse.ok)`.

JSON casse : `reponse.json()` rejette. Encore le `catch`.

Bug dans ton affichage : aussi le `catch`, si tu as enveloppe assez large. Parfois trop large. Apprends a lire `console.error` pour voir la vraie cause.

Ne mens pas a l'utilisateur

"Erreur 500 Internal Server Error" ne parle qu'aux devs. Preferes "Le service produits est indisponible. Reessaie dans cinq minutes." Tu peux garder le detail pour la console ou un mode debug.

Pour un formulaire de contact, "Envoi impossible (reseau)" vaut mieux qu'un ecran blanc.

Cas meteo

```
async function meteo(ville) {  
  try {  
    const reponse = await fetch(  
      "https://exemple.api/meteo?ville=" + encodeURIComponent(ville)  
    );  
    if (!reponse.ok) {  
      throw new Error("ville ou service introuvable");  
    }  
    const data = await reponse.json();  
    return data;  
  } catch (e) {  
    return null;  
  }  
}  
  
async function ui() {  
  const data = await meteo("Toulouse");  
  const zone = document.querySelector("#meteo");  
  if (!data) {  
    zone.textContent = "Pas de meteo pour cette ville.";  
    return;  
  }  
  zone.textContent = data.temp + "°C - " + data.ciel;  
}
```

Renvoyer `null` puis tester, c'est une autre facon de gerer. L'important : ne jamais faire comme si les donnees etaient la.

Erreur classique

Afficher "Succes !" alors que `ok` est false, parce que tu n'as regarde que "pas d'exception". Ou avaler l'erreur avec un `catch` vide. Un `catch` vide, c'est cacher la poussiere sous le tapis.

En vrai

Force une erreur. URL inventee. Puis une URL qui existe mais renvoie 404. Observe la difference entre rejet reseau et `ok === false`. Ecris les deux cas dans un petit commentaire pour toi.

A toi

Prends ton chargeur de liste. Ajoute `response.ok`, un vrai message utilisateur, et un `console.error`. Teste le chemin heureux et le chemin rate. Les deux doivent "marcher" : l'un affiche des donnees, l'autre explique le probleme.

Chapitre 8 - fetch POST : envoyer un peu de JSON

GET, c'est "donne-moi". POST, c'est "voici quelque chose". Formulaire de contact, creation d'une todo, envoi d'un avis produit : souvent, tu POSTES un corps JSON.

On reste léger. Pas besoin d'auth compliquée. Juste : méthode, headers, body.

Anatomie d'un POST

```
async function envoyerContact(contact) {
  const reponse = await fetch("https://exemple.api/contact", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify(contact)
  });

  if (!reponse.ok) {
    throw new Error("Envoi refuse (" + reponse.status + ")");
  }

  return reponse.json(); // parfois le serveur renvoie { ok: true, id: 12 }
}
```

Trois ingrédients. `method: "POST"` pour dire que tu envoies. `Content-Type: application/json` pour prévenir que le corps est du JSON. `body` avec le texte produit par `JSON.stringify`.

Sans le header, certains serveurs ne comprennent pas. Sans `stringify`, tu envoies "[object Object]" et ça finit mal.

Brancher sur un formulaire

```
const form = document.querySelector("#contact");
const statut = document.querySelector("#statut");

form.addEventListener("submit", async function (event) {
  event.preventDefault();

  const contact = {
    nom: form.nom.value.trim(),
    email: form.email.value.trim(),
    message: form.message.value.trim()
  };

  if (contact.nom === "" || contact.message.length < 10) {
    statut.textContent = "Complete le formulaire avant d'envoyer.";
    return;
  }

  statut.textContent = "Envoi en cours...";

  try {
    const resultat = await envoyerContact(contact);
    statut.textContent = "Message envoye. Merci !";
    form.reset();
    console.log(resultat);
  }
});
```



```
    } catch (e) {  
      statut.textContent = "Envoi impossible. Verifie ta connexion.";  
      console.error(e);  
    }  
  });
```

Tu valides d'abord (chapitre 2). Tu montres un etat intermediaire. Tu postes. Tu geres succes et echec. C'est un flux realiste de page contact chez DanielCraft et partout ailleurs.

GET vs POST en une phrase

GET pour lire sans modifier (liste, meteo, detail produit). POST pour creer ou envoyer (message, nouvelle tache, commande). Il existe aussi PUT, PATCH, DELETE. Plus tard. Pour ce livre, GET + POST suffisent largement.

Ce que le serveur renvoie

Parfois rien d'utile (204). Parfois un objet `{ "id": 42, "status": "recu" }`. Parfois une erreur JSON `{ "erreur": "email invalide" }`. Lis la doc de l'API quand tu en as une. En exercice, tu peux simuler avec un service de test ou un petit backend. L'important cote front : savoir envoyer proprement.

Erreur classique

Ecrire `body: contact` au lieu de `body: JSON.stringify(contact)`. Ou oublier `async` sur le handler `submit` et mettre `await` dedans. Ou poster sans `preventDefault` et perdre l'etat au rechargement.

Autre piege : croire que POST "marche" parce que le reseau n'a pas plante, sans lire `reponse.ok`. Meme regle qu'au chapitre 7.

En vrai

Si tu as un endpoint de test (ou JSONPlaceholder style posts), envoie un petit objet `{ titre, corps }`. Affiche l'id renvoye. Sinon, ecris la fonction et un `console.log(JSON.stringify(contact))` pour verifier le corps avant meme d'appeler le reseau.

A toi

Ajoute un bouton "Envoyer l'avis" avec note + commentaire. Valide. POST en JSON. Affiche "Avis enregistre" ou un message d'erreur. Garde le code court. La clarte bat la longueur.

Chapitre 9 - Modules : decouper le code



Un fichier = une responsabilite. import / export.

Quand ton fichier `app.js` fait 400 lignes, tu te perds. Tu cherches une fonction pendant cinq minutes. Tu as peur de casser quelque chose en touchant une ligne. Les modules existent pour ca : un fichier, une responsabilite.

En JavaScript moderne, tu ecris `export` pour sortir une fonction d'un fichier, et `import` pour la recuperer ailleurs. Dans le HTML, tu charges ton point d'entree avec `type="module"`.

```
<script type="module" src="main.js"></script>
```

Un exemple simple

Fichier `api.js` :

```
export async function chargerListe(url) {
  const reponse = await fetch(url);
  if (!reponse.ok) throw new Error("Chargement impossible");
  return reponse.json();
}
```

Fichier `main.js` :

```
import { chargerListe } from "./api.js";

const produits = await chargerListe("/api/produits.json");
console.log(produits);
```

Tu vois l'idee. `main.js` orchestre. `api.js` parle au reseau. Plus tard, tu peux ajouter `afficher.js` pour le DOM. Chaque fichier reste court.

Pieges a connaitre

Les modules fonctionnent mieux avec un petit serveur local (pas toujours en ouvrant le fichier en `file://`). Si tu testes avec Live Server, VS Code, ou `python -m http.server`, tu es bien.

Les chemins relatifs comptent : `./api.js` veut dire "dans le meme dossier". Oublie le `./` et certains navigateurs ralentissent ou se plaignent.

N'exporte que ce qui est utile. Tout exporter "au cas ou" reforge le desordre.

En vrai

Sur un vrai projet, tu n'as pas besoin de vingt fichiers le premier jour. Trois fichiers clairs battent un monstre de 800 lignes. DanielCraft te conseille de découper des que tu recopies la meme fonction deux fois.

A toi

Prends un vieux script unique. Identifie deux fonctions que tu pourrais sortir dans un autre fichier. Ecris mentalement leurs noms d'export. Tu n'as pas besoin de tout migrer maintenant. Juste sentir la decoupe.

Chapitre 10 - Organiser un petit projet

Un projet web, ce n'est pas seulement "du code qui marche". C'est aussi un rangement que tu comprends dans une semaine, quand tu auras oublié les détails.

Voici une structure simple qui marche pour beaucoup de mini-apps :

```
mon-projet/  
  index.html  
  styles.css  
  main.js  
  api.js  
  afficher.js  
  data/ (optionnel)
```

`index.html` : la page. Peu de logique. `styles.css` : l'habillage. `main.js` : le chef d'orchestre (écoute les clics, lance les chargements). `api.js` : fetch et parsing. `afficher.js` : créer des éléments DOM à partir des données.

Responsabilités

Si `api.js` commence à manipuler le DOM, tu mélanges les rôles. Si `afficher.js` appelle `fetch`, pareil. Garde une règle : chaque fichier a une phrase pour se présenter. "Moi, je charge les données." "Moi, je dessine la liste." "Moi, je branche les boutons."

Noms clairs

`faireTruc.js` n'aide personne. `chargerProduits`, `afficherErreur`, `viderListe` : on comprend. Les noms longs et clairs battent les noms courts et mystérieux.

Une seule source de vérité

Évite d'avoir le même tableau de produits copié dans trois endroits. Charge une fois, stocke dans une variable (ou un petit état), puis affiche. Si tu modifies, tu modifies à un seul endroit.

En vrai

Beaucoup de débutants collent tout dans `index.html` entre deux balises script. Ça marche pour dix lignes. Au-delà, c'est une nasse. Passe tôt aux fichiers séparés, même pour un exercice.

A toi

Dessine sur papier (ou dans un commentaire) les 4 fichiers de ton prochain mini-projet et une phrase de rôle pour chacun. Si tu ne trouves pas la phrase, le fichier n'est peut-être pas nécessaire.

Chapitre 11 - Deboguer mieux

Quand ca casse, le reflexe "je recois tout" est humain. Le reflexe utile, c'est : lire le message, trouver la ligne, verifier une hypothese.

La console, ton amie

`console.log` n'est pas honteux. C'est un projecteur. Affiche la valeur juste avant l'endroit douteux. Affiche `reponse.status`. Affiche `typeof data`. Souvent, le bug saute aux yeux : `undefined`, tableau vide, mauvaise cle JSON.

Pense aussi a `console.error` pour les vrais problemes, et a retirer les logs inutiles avant de montrer ton travail.

Lire une stack

Quand le navigateur affiche une erreur rouge, il donne souvent un fichier et un numero de ligne. Clique dessus. Regarde la pile : "appelle depuis telle fonction, elle-meme depuis telle autre". Tu remontes le fil.

Si le message dit `Cannot read properties of null`, tu as probablement selectionne un element qui n'existe pas encore, ou un mauvais selecteur.

Points d'arret

Dans les outils developpeur (F12), onglet Sources, tu peux cliquer a gauche d'une ligne pour poser un breakpoint. La page s'arrete la. Tu inspectes les variables. Tu avances pas a pas. C'est plus precis qu'un mur de logs.

Hypotheses courtes

Ecris (mentalement) : "Je pense que `data` n'est pas un tableau." Puis verifie. Une hypothese a la fois. Sinon tu changes trois choses et tu ne sais plus ce qui a marche.

En vrai

Les developpeurs experimentes passent une bonne partie du temps a lire des erreurs. Ce n'est pas un echec. C'est le metier. DanielCraft te le dit franchement : savoir deboguer compte autant que savoir ecrire.

A toi

Prends un vieux bug (ou invente-en un : mauvais id HTML). Force l'erreur. Lis le message complet a voix haute. Relie-le a une ligne precise. C'est l'entrainement.

Chapitre 12 - Mini-projet : liste depuis une API



Mini-projet : recuperer des donnees et les montrer proprement.

On assemble. Objectif : une page qui charge une liste (produits, citations, ou posts fictifs) depuis une URL JSON, affiche les titres, et gere le cas d'erreur proprement.

Tu peux utiliser une API publique de demo, ou un fichier `produits.json` servi en local. L'important n'est pas la source exacte. C'est le parcours.

Parcours

1. HTML : un titre, un bouton "Charger", une zone `#liste`, une zone `#message`.
2. Au clic, tu appelles une fonction `async` qui fait `fetch`.
3. Tu verifies `reponse.ok`.
4. Tu parses le JSON.
5. Tu vides `#liste`, puis tu ajoutes un element par item.
6. Si ca rate, tu ecris un message humain dans `#message` ("Impossible de charger. Reessaie.").

Squelette

```
async function charger() {
  const message = document.querySelector("#message");
  const liste = document.querySelector("#liste");
  message.textContent = "Chargement...";
  try {
    const reponse = await fetch("./produits.json");
    if (!reponse.ok) throw new Error("HTTP " + reponse.status);
    const data = await reponse.json();
    liste.innerHTML = "";
    for (const item of data) {
      const li = document.createElement("li");
      li.textContent = item.nom;
      liste.appendChild(li);
    }
    message.textContent = data.length + " elements";
  } catch (e) {
    message.textContent = "Impossible de charger. Reessaie.";
    console.error(e);
  }
}
```

Variante meteo (idee)

Meme structure : bouton, chargement, affichage temperature ou description, erreur claire. L'API change. Le schema mental reste.

Qualite minimale

Pas de page blanche silencieuse. Pas d'erreur technique crachee a l'utilisateur. Un etat "chargement" visible. Des noms de variables lisibles.

A toi

Code cette page en local. Casse volontairement l'URL. Verifie que ton message d'erreur apparait. Remets l'URL. Verifie la liste. Si ces deux chemins marchent, le mini-projet est reussi.

Chapitre 13 - Ce qu'il faut retenir

Tu as grimpe d'un cran. Voici la carte en version poche.

En une minute

Les formulaires : valider, `preventDefault`, messages utiles. JSON : `parse` et `stringify` pour echanger des donnees. `fetch` : demander au serveur. Promesses puis `async/await` : attendre sans embrouiller le fil. Erreurs : `response.ok` + `try/catch`. POST : envoyer un body JSON quand il faut. Modules : decouper. Organisation : un role par fichier. Debug : lire la ligne, tester une hypothese.

Habitudes solides

1. Toujours gerer l'echec reseau
2. Montrer un etat de chargement
3. Messages humains a l'ecran, details techniques en console
4. Fichiers courts
5. Noms clairs
6. Un changement a la fois quand tu debogues

Erreurs classiques

Oublier `await`. Oublier de verifier `ok`. Parser du HTML en croyant que c'est du JSON. Melanger DOM et fetch dans le meme monstre de fichier. Tester en `file://` avec des modules et ne pas comprendre pourquoi ca bloque.

Suite dans ce livre

Ateliers pratiques, puis CORS, debounce, perf legere, quiz, bravo. Tu n'as pas besoin d'etre parfait. Tu as besoin d'etre capable de charger, afficher, et expliquer ce qui casse.

Mini defi

Sans regarder tes notes, ecris de memoire le squelette `async` + `fetch` + `ok` + `json` + `catch`. Compare ensuite. Les trous montrent ce qu'il faut relire.

Chapitre 14 - Atelier : formulaire robuste

Objectif : un formulaire contact (nom, email, message) qui refuse l'envoi si un champ manque, et qui explique clairement quoi corriger.

Etapas

1. Cree `index.html` avec un formulaire, trois champs, un bouton, une zone `#erreurs`.
2. Ecoute `submit`. Appelle `preventDefault()` tout de suite.
3. Lis les valeurs avec `.value.trim()`.
4. Construis une liste d'erreurs (tableau de phrases).
5. Si le nom est vide : "Indique ton nom."
6. Si l'email n'a pas de `@` : "Email incomplete."
7. Si le message a moins de 10 caracteres : "Message trop court."
8. Affiche les erreurs dans `#erreurs`. Si aucune erreur, affiche "Formulaire pret (simulation d'envoi)."

Criteres de reussite

- Sans JS, le navigateur ne doit pas naviguer nulle part a cause du submit (donc `preventDefault` marche).
- Les messages sont en francais simple.
- Tu peux envoyer seulement quand tout est valide.
- Tu logs en console l'objet `{ nom, email, message }` une fois valide (simulation).

Bonus

Desactive le bouton pendant une fausse attente d'une seconde (`setTimeout`), puis reactive-le. Ca prepare le vrai envoi reseau.

Piege

Ne te contente pas de `alert()`. Les alertes agacent. Un message dans la page est plus pro, meme pour un exercice.

Chapitre 15 - Atelier : fetch et afficher

Objectif : charger un JSON et remplir une liste HTML.

Preparation

Cree un fichier `citations.json` a cote de ta page :

```
[
  { "texte": "Petit a petit, l'oiseau fait son nid.", "auteur": "Proverbe" },
  { "texte": "On apprend en faisant.", "auteur": "DanielCraft" },
  { "texte": "Le code clair se lit comme une histoire.", "auteur": "Atelier" }
]
```

Sers le dossier avec un petit serveur local.

Etapes

1. Bouton "Charger les citations".
2. `fetch("./citations.json")`.
3. Verifie `ok`, puis `json()`.
4. Pour chaque item, cree un `<li>` avec le texte et l'auteur.
5. Affiche "Chargement..." pendant l'attente.
6. En cas d'erreur, message humain + `console.error`.

Criteres de reussite

- Trois citations visibles.
- URL cassee volontairement -> message d'erreur.
- Pas d'exception non geree dans la console (sauf ton log volontaire).

Bonus

Ajoute un champ recherche qui filtre la liste deja chargee (sans refaire fetch a chaque lettre pour l'instant).
Le debounce viendra plus tard.

Chapitre 16 - Atelier : decouper en modules

Objectif : reprendre l'atelier fetch et le decouper en trois fichiers.

Fichiers

- `api.js` : `export async function chargerCitations()`
- `afficher.js` : `export function afficherCitations(liste, data)`
- `main.js` : branche le bouton, appelle api + afficher, gere le message d'erreur

HTML :

```
<script type="module" src="main.js"></script>
```

Etapas

1. 1. Deplace le `fetch` dans `api.js`.
2. 2. Deplace la boucle DOM dans `afficher.js`.
3. 3. Dans `main.js`, importe les deux fonctions.
4. 4. Garde le `try/catch` dans `main.js` (orchestration).

Criteres de reussite

- Ca marche comme avant.
- Aucun fichier ne depasse ~40 lignes (ordre de grandeur).
- Tu peux expliquer a voix haute le role de chaque fichier en une phrase.

Bonus

Ajoute `export function afficherErreur(zone, texte)` dans `afficher.js`. `main.js` ne touche plus au DOM des messages directement.

Chapitre 17 - CORS, en une page claire

Tu appelles une API. La console crie quelque chose avec "CORS". Tu te dis que JavaScript est casse. Souvent, non : c'est le navigateur qui protege.

L'idée simple

Une page hebergée sur `http://localhost:3000` qui demande des données à `https://autre-site.com` fait une requête "inter-origine". Le navigateur demande au serveur distant s'il accepte. Si le serveur ne dit pas oui (en-têtes CORS), le navigateur bloque la réponse côté page.

Ce n'est pas toi qui "codes mal" à chaque fois. C'est une règle de sécurité : un site malveillant ne doit pas lire facilement les données privées d'un autre site à ta place.

Ce que tu peux faire en apprenant

- Utilise des API qui autorisent le navigateur (demos publiques).
- Ou passe par ton propre serveur (backend) qui, lui, appelle l'API.
- Ou travaille avec des fichiers JSON locaux pendant les exercices.

Ce qu'il ne faut pas croire

"Désactiver CORS" dans le navigateur n'est pas une solution pro. Ça peut aider cinq minutes pour un test, ça ne règle rien pour tes utilisateurs.

En vrai

Quand tu verras CORS, lis le message : origine bloquée, méthode, en-têtes. Puis demande : est-ce que je suis censé appeler cette API depuis le navigateur, ou depuis un serveur ? Cette question évite des heures de panique.

A toi

Note en une phrase ta définition personnelle de CORS. Si tu peux l'expliquer à un ami sans jargon, c'est gagné.

Chapitre 18 - Debounce : ne pas frapper trop vite

Imagine une barre de recherche. A chaque lettre, tu lances un `fetch`. L'utilisateur tape "bonsoir" : sept requetes. Le serveur souffle. L'affichage clignote. Mauvaise idee.

Le debounce, c'est attendre un petit silence avant d'agir. Tu tapes, tu tapes, tu pauses 300 ms : la seulement, tu lances la recherche.

Idee en code

```
function debounce(fn, delai) {
  let timer;
  return function (...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delai);
  };
}

const rechercher = debounce((texte) => {
  console.log("Je cherche :", texte);
  // ici : fetch ou filtre local
}, 300);

input.addEventListener("input", (e) => rechercher(e.target.value));
```

Quand l'utiliser

Recherche pendant la frappe. Redimensionnement de fenetre. Sauvegarde auto. Partout ou un evenement se repete trop vite.

Quand ne pas l'utiliser

Un clic sur un bouton "Envoyer" : pas besoin. Un compteur de jeu a chaque frame : ce n'est pas le meme outil.

En vrai

300 ms est un bon depart pour une recherche. Trop court : encore trop de requetes. Trop long : l'interface semble lente. Ajuste selon le feeling.

A toi

Ajoute un debounce sur un champ input qui `console.log` la valeur. Tape vite. Verifie que le log n'apparait qu'apres une pause.

Chapitre 19 - Petites habitudes de performance

Pas besoin de devenir expert perf pour éviter les bêtises coûteuses. Quelques gestes changent déjà beaucoup.

Moins de travail inutile

Ne recrée pas toute la liste HTML si un seul titre change. Ne relance pas un fetch à chaque pixel de scroll. Ne charge pas une image géante pour une icône.

Dom : batch mental

Quand tu ajoutes 50 éléments, construis-les puis ajoute-les (ou utilise un fragment). Évite d'alterner lecture et écriture du DOM dans une boucle serrée si tu n'y es pas obligé : ça peut ralentir.

Reseau

Cache ce qui ne change pas pendant la session si c'est pertinent. Affiche un squelette ou "Chargement..." pour que l'attente soit digeste. Gère les erreurs au lieu de laisser tourner indéfiniment.

Code lisible d'abord

Optimiser un code confus, c'est peindre une voiture sans moteur. D'abord clair. Ensuite mesure. Ensuite optimise le vrai goulot. Les outils navigateur (Performance, Network) existent pour ça.

En vrai

Pour les projets de ce livre, la perf "suffisante" c'est : réponse humaine, pas de boucle folle, pas de spam réseau. DanielCraft préfère un app simple et stable à une app "optimisée" illisible.

A toi

Relis ton mini-projet. Cherche un seul gaspillage (fetch en double, logs en rafale, grosse image). Corrige ce seul point.

Quiz final

Pas de piège. Relis une fois si besoin. L'idée : vérifier que tu peux avancer sans paniquer.

Questions

1. 1. Pourquoi appelle-t-on souvent `preventDefault()` sur un submit de formulaire ?

- A) Pour colorier le bouton
- B) Pour empêcher l'envoi / rechargement par défaut du navigateur
- C) Pour vider le localStorage

1. 2. `JSON.stringify` sert surtout à :

- A) Transformer un objet JS en texte JSON
- B) Afficher une image
- C) Créer un fichier CSS

1. 3. Que fait `await` devant un `fetch` ?

- A) Il ignore la réponse
- B) Il attend la réponse avant de continuer dans la fonction async
- C) Il compile le HTML

1. 4. Pourquoi vérifier `reponse.ok` ?

- A) Parce que fetch ne lève pas toujours d'erreur sur un 404
- B) Parce que JSON est obligatoire
- C) Parce que le DOM l'exige

1. 5. Dans un module, `export` sert à :

- A) Supprimer une fonction
- B) Rendre une fonction (ou valeur) importable ailleurs
- C) Lancer le serveur

1. 6. CORS, en une idée :

- A) Un langage de programmation
- B) Une règle navigateur sur les requêtes entre origines différentes
- C) Un type de variable

1. 7. Le debounce sert surtout à :

- A) Attendre un petit silence avant d'agir (ex: recherche)
- B) Chiffrer les mots de passe
- C) Remplacer CSS

1. 8. Où placer le `try/catch` autour d'un chargement async ?

- A) Autour de l'appel `await` / logique de chargement

- B) Uniquement autour de `console.log`
- C) Autour de la balise `<html>`

1. 9. Pour charger un script module dans HTML, on écrit souvent :

- A) `<script type="module" src="main.js">`
- B) `<script type="css" src="main.js">`
- C) `<module href="main.js">`

1. 10. Un bon message d'erreur utilisateur ressemble à :

- A) `TypeError: undefined is not a function`
- B) Impossible de charger. Réessaie dans un instant.
- C) Une page blanche

1. 11. `response.json()` renvoie :

- A) Directement un nombre magique
- B) Une promesse qui donne les données parsees
- C) Un fichier PDF

1. 12. Organiser un projet, règle simple :

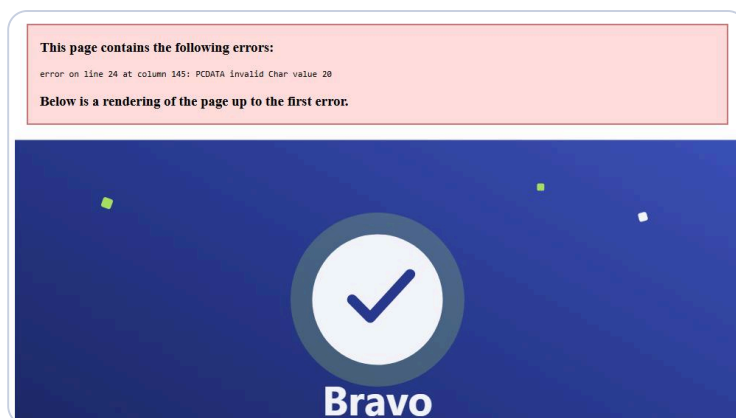
- A) Tout dans un seul fichier de 2000 lignes
- B) Un rôle clair par fichier
- C) Copier-coller le même fetch partout

Corriges

1-B, 2-A, 3-B, 4-A, 5-B, 6-B, 7-A, 8-A, 9-A, 10-B, 11-B, 12-B.

Si tu as 9/12 ou plus : tu es prêt pour les petits projets réels. Sinon, relis les chapitres liés aux erreurs, sans dramatiser.

Bravo



Bravo. Tu sais faire parler ta page avec le monde extérieur.

Tu as fini JavaScript - La suite. Chez DanielCraft, on marque le moment : tu ne fais plus seulement réagir un bouton. Tu sais valider un formulaire, parler JSON, appeler un serveur, attendre proprement, découper en modules, et lire une erreur sans fondre.

Ce que tu sais faire maintenant

Charger des données. Afficher une liste. Dire à l'utilisateur quand ça charge et quand ça rate. Envoyer un POST simple. Ranger ton code. Ralentir une recherche avec un debounce. Comprendre CORS sans mystère total.

La suite possible

Frameworks (React, Vue, Svelte...) plus tard, si tu veux. TypeScript, tests, bundlers... aussi. Mais tu as le socle qui porte tout ça : fetch, async, structure. Sans ce socle, un framework devient de la magie opaque.

Un conseil pour la route

Fais une mini-app perso cette semaine. Même petite. Même moche. Liste de recettes, citations, suivi d'habitudes. Le muscle se construit en publiant des tentatives, pas en collectionnant des tutoriels.

A toi

Ecris en trois lignes : ton prochain projet, la donnée que tu vas charger, et le premier fichier que tu vas créer. Puis ouvre l'éditeur.

Bravo. Vraiment.

Tu as les briques. Maintenant, construis.